

# Main.java explicado linha por linha

Projeto Carbo · cada pedaço do código e o nome que ele tem em Java

## O código completo

```
1  import java.util.List;
2
3  public class Main {
4
5      // Um alimento: nome + gramas de carboidrato a cada 100 g.
6      record Alimento(String nome, double carboidratosPor100g) {
7      }
8
9      // Valores aproximados da Tabela TACO.
10     static final List<Alimento> LISTA_DE_ALIMENTOS = List.of(
11         new Alimento("Arroz branco cozido", 28.1),
12         new Alimento("Arroz integral cozido", 25.8),
13         new Alimento("Feijão carioca cozido", 13.6),
14         new Alimento("Feijão preto cozido", 14.0),
15         new Alimento("Peito de frango grelhado", 0.0),
16         new Alimento("Ovo de galinha cozido", 0.6),
17         new Alimento("Batata doce cozida", 18.4),
18         new Alimento("Batata inglesa cozida", 11.9),
19         new Alimento("Pão francês", 58.6),
20         new Alimento("Banana prata", 26.0),
21         new Alimento("Maçã fuji", 15.2));
22
23     public static void main(String[] args) {
24         for (Alimento item : LISTA_DE_ALIMENTOS) {
25             System.out.println(item.nome() + " - " + item.carboidratosPor100g() + "
26             g");
27         }
28     }
```

## Linha 1: a importação

```
import java.util.List;
```

| Pedaço                 | O que é                                                                                                                |
|------------------------|------------------------------------------------------------------------------------------------------------------------|
| <code>import</code>    | <b>Palavra-chave</b> (palavra reservada da linguagem). Traz algo de outro lugar para este arquivo usar.                |
| <code>java.util</code> | <b>Pacote</b> : uma "pasta" de código que já vem com o Java. O <code>util</code> reúne utilidades como listas e datas. |
| <code>List</code>      | <b>Interface</b> (um tipo de molde) que representa uma lista.                                                          |
| <code>;</code>         | <b>Fim da instrução</b> . Em Java, toda instrução termina com ponto e vírgula.                                         |

Sem essa linha, o Java não saberia o que é `List` na linha 10.

## Linha 3: a classe

```
public class Main {
```

| Pedaço              | O que é                                                                                                                              |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>public</code> | <b>Modificador de acesso</b> : qualquer parte do programa pode usar essa classe.                                                     |
| <code>class</code>  | <b>Palavra-chave</b> que declara uma <b>classe</b> , o molde principal do Java. Todo código Java precisa morar dentro de uma classe. |
| <code>Main</code>   | <b>Nome da classe</b> . Por convenção, começa com maiúscula e é igual ao nome do arquivo ( <code>Main.java</code> ).                 |
| <code>{</code>      | <b>Abre o bloco</b> da classe. Tudo até o <code>}</code> da linha 28 pertence a ela.                                                 |

## Linhas 5 e 9: comentários

```
// Um alimento: nome + gramas de carboidrato a cada 100 g.
```

| Pedaço          | O que é                                                                                    |
|-----------------|--------------------------------------------------------------------------------------------|
| <code>//</code> | <b>Comentário de linha</b> . O Java ignora tudo depois disso. É uma anotação para humanos. |

## Linhas 6–7: o record

```
record Alimento(String nome, double carboidratosPor100g) {  
}
```

| Pedaço                           | O que é                                                                                                                                           |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>record</code>              | <b>Palavra-chave</b> que declara um <b>record</b> : um tipo especial de classe feito só para guardar dados.                                       |
| <code>Alimento</code>            | <b>Nome do record</b> . É o <b>molde</b> (o <b>tipo</b> ) de todo alimento. Como está dentro de <code>Main</code> , é um <b>record aninhado</b> . |
| <code>( ... )</code>             | <b>Lista de componentes</b> do record, ou seja, os dados que todo <code>Alimento</code> precisa ter.                                              |
| <code>String</code>              | <b>Tipo de referência</b> : guarda texto. Uma <code>String</code> também é um objeto.                                                             |
| <code>nome</code>                | <b>Componente</b> (ou <b>campo</b> ) do record, do tipo <code>String</code> .                                                                     |
| <code>,</code>                   | Separa um componente do outro.                                                                                                                    |
| <code>double</code>              | <b>Tipo primitivo</b> : guarda número com casas decimais. "Primitivo" quer dizer que é um valor simples, não um objeto.                           |
| <code>carboidratosPor100g</code> | <b>Componente</b> (campo) do record, do tipo <code>double</code> .                                                                                |
| <code>{ }</code>                 | <b>Corpo</b> do record. Está vazio porque não precisamos acrescentar nada.                                                                        |

Por baixo dos panos, o Java **gera sozinho** para o record:

- um **construtor**, que é o que o `new Alimento(...)` chama para criar o objeto;
- os **métodos de acesso** `nome()` e `carboidratosPor100g()`, usados na linha 25;
- o `toString()`, que transforma o objeto em texto, além de `equals()` e `hashCode()`, que comparam objetos.

## Linha 10: a constante

```
static final List<Alimento> LISTA_DE_ALIMENTOS = List.of(
```

| Pedaco                            | O que é                                                                                                                                                                |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>static</code>               | <b>Modificador:</b> o dado pertence à <b>classe</b> <code>Main</code> e não a um objeto. Existe uma única cópia, e dá para usar sem criar nada com <code>new</code> .  |
| <code>final</code>                | <b>Modificador:</b> depois de receber um valor, não pode ser <b>reatribuída</b> . É o equivalente ao <code>const</code> do JavaScript.                                 |
| <code>static final</code> juntos  | Formam uma <b>constante</b> .                                                                                                                                          |
| <code>List&lt;Alimento&gt;</code> | <b>Tipo</b> da constante: uma lista.                                                                                                                                   |
| <code>&lt;Alimento&gt;</code>     | <b>Parâmetro de tipo</b> (ou <b>genérico</b> ): diz que essa lista só aceita objetos <code>Alimento</code> . Se você tentar colocar um texto, o Java recusa.           |
| <code>LISTA_DE_ALIMENTOS</code>   | <b>Nome da constante</b> , em MAIÚSCULAS por convenção.                                                                                                                |
| <code>=</code>                    | <b>Operador de atribuição:</b> "recebe o valor de".                                                                                                                    |
| <code>List</code>                 | A <b>interface</b> importada na linha 1.                                                                                                                               |
| <code>.</code>                    | <b>Operador de acesso:</b> "entra em" algo para usar o que tem lá dentro.                                                                                              |
| <code>of(...)</code>              | <b>Método estático</b> da <code>List</code> , que cria uma lista com os itens passados. Essa lista é <b>imutável</b> : não dá para adicionar nem remover itens depois. |

## Linhas 11–21: os objetos

```
new Alimento("Arroz branco cozido", 28.1),
```

| Pedaco                                      | O que é                                                                                                                                             |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>new</code>                            | <b>Operador</b> que cria um <b>objeto</b> novo.                                                                                                     |
| <code>Alimento(...)</code>                  | <b>Chamada ao construtor</b> do record, o "use a forma" da analogia do bolo.                                                                        |
| <code>"Arroz branco cozido"</code>          | <b>Literal de String:</b> um texto escrito direto no código, entre aspas duplas. É o <b>argumento</b> que vai para o componente <code>nome</code> . |
| <code>28.1</code>                           | <b>Literal double:</b> um número escrito direto no código. É o <b>argumento</b> que vai para <code>carboidratosPor100g</code> .                     |
| <code>,</code> (no fim)                     | Separa um item da lista do próximo.                                                                                                                 |
| <code>)</code> e <code>;</code> da linha 21 | O <code>)</code> fecha o <code>List.of()</code> da linha 10 e o <code>;</code> termina a instrução iniciada lá.                                     |

São **11 objetos** (ou **instâncias**) do tipo `Alimento`, e todos vão direto para dentro da lista.

**Argumento vs parâmetro:** o **parâmetro** é a "vaga" declarada no molde ( `String nome` ). O **argumento** é o valor concreto que você passa para essa vaga ( `"Arroz branco cozido"` ).

## Linha 23: o método main

```
public static void main(String[] args) {
```

| Pedaço                | O que é                                                                                                                                                                                  |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>public</code>   | <b>Modificador de acesso:</b> o Java precisa conseguir chamar esse método de fora.                                                                                                       |
| <code>static</code>   | Pertence à classe. O Java chama o <code>main</code> sem criar nenhum objeto <code>Main</code> .                                                                                          |
| <code>void</code>     | <b>Tipo de retorno:</b> "nada". O método faz algo, mas não devolve nenhum valor.                                                                                                         |
| <code>main</code>     | <b>Nome do método.</b> É o <b>ponto de entrada</b> do programa: quando você roda <code>java Main.java</code> , é por aqui que tudo começa. Esse nome é obrigatório.                      |
| <code>String[]</code> | <b>Tipo array de String.</b> Os <code>[]</code> indicam um array, uma sequência de tamanho fixo.                                                                                         |
| <code>args</code>     | <b>Parâmetro</b> do método: recebe palavras extras digitadas depois de <code>java Main.java</code> no terminal. O seu código não usa, mas a assinatura do <code>main</code> precisa ter. |
| <code>{</code>        | <b>Abre o bloco</b> do método.                                                                                                                                                           |

## Linha 24: o laço

```
for (Alimento item : LISTA_DE_ALIMENTOS) {
```

| Pedaço                          | O que é                                                                                                                                 |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <code>for</code>                | <b>Palavra-chave de laço</b> (loop): repete um bloco de código. Esse formato com <code>:</code> se chama <b>for-each</b> , "para cada". |
| <code>Alimento</code>           | <b>Tipo</b> da variável do laço.                                                                                                        |
| <code>item</code>               | <b>Variável local do laço.</b> A cada volta, ela recebe um dos 11 objetos e só existe dentro das <code>{ }</code> do <code>for</code> . |
| <code>:</code>                  | Lê-se " <b>em</b> ": "para cada <code>item</code> <b>em</b> <code>LISTA_DE_ALIMENTOS</code> ".                                          |
| <code>LISTA_DE_ALIMENTOS</code> | A <b>constante</b> da linha 10, que é a coleção percorrida.                                                                             |

## Linha 25: a impressão

```
System.out.println(item.nome() + " - " + item.carboidratosPor100g() + " g");
```

| Pedaço                                  | O que é                                                                                                                                               |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>System</code>                     | <b>Classe</b> que já vem com o Java e representa o "sistema" onde o programa roda.                                                                    |
| <code>.out</code>                       | <b>Campo estático</b> de <code>System</code> : um <b>objeto</b> que representa a saída do terminal.                                                   |
| <code>.println(...)</code>              | <b>Método</b> que imprime um texto e pula uma linha ("print line").                                                                                   |
| <code>item.nome()</code>                | <b>Chamada do método de acesso</b> <code>nome()</code> no objeto <code>item</code> . Devolve o nome, por exemplo <code>"Arroz branco cozido"</code> . |
| <code>+</code>                          | <b>Operador de concatenação</b> : com texto, o <code>+</code> <b>junta</b> em vez de somar.                                                           |
| <code>" - " e " g"</code>               | <b>Literais de String</b> .                                                                                                                           |
| <code>item.carboidratosPor100g()</code> | <b>Chamada do método de acesso</b> : devolve o <code>double</code> , por exemplo <code>28.1</code> . Ao ser concatenado, vira texto.                  |
| <code>;</code>                          | <b>Fim da instrução</b> .                                                                                                                             |

Resultado de uma volta: `Arroz branco cozido - 28.1 g`

## Linhas 26–28: fechamentos

| Linha | O } fecha                           |
|-------|-------------------------------------|
| 26    | O bloco do <code>for</code>         |
| 27    | O bloco do método <code>main</code> |
| 28    | O bloco da classe <code>Main</code> |

## Glossário

| Termo                              | No seu código                                                                                                                                                                             |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Palavra-chave</b>               | <code>import</code> , <code>public</code> , <code>class</code> , <code>record</code> , <code>static</code> , <code>final</code> , <code>new</code> , <code>void</code> , <code>for</code> |
| <b>Pacote</b>                      | <code>java.util</code>                                                                                                                                                                    |
| <b>Interface</b>                   | <code>List</code>                                                                                                                                                                         |
| <b>Classe</b>                      | <code>Main</code> , <code>System</code>                                                                                                                                                   |
| <b>Record</b> (classe só de dados) | <code>Alimento</code>                                                                                                                                                                     |
| <b>Componente / campo</b>          | <code>nome</code> , <code>carboidratosPor100g</code>                                                                                                                                      |
| <b>Tipo primitivo</b>              | <code>double</code>                                                                                                                                                                       |
| <b>Tipo de referência</b>          | <code>String</code> , <code>List&lt;Alimento&gt;</code> , <code>Alimento</code> , <code>String[]</code>                                                                                   |
| <b>Genérico</b>                    | <code>&lt;Alimento&gt;</code>                                                                                                                                                             |
| <b>Constante</b>                   | <code>LISTA_DE_ALIMENTOS</code>                                                                                                                                                           |
| <b>Objeto / instância</b>          | Os 11 <code>new Alimento(...)</code> , e também <code>System.out</code>                                                                                                                   |
| <b>Construtor</b>                  | <code>Alimento(...)</code> , chamado pelo <code>new</code>                                                                                                                                |
| <b>Método</b>                      | <code>main</code> , <code>List.of</code> , <code>println</code> , <code>nome()</code> , <code>carboidratosPor100g()</code>                                                                |
| <b>Método estático</b>             | <code>main</code> , <code>List.of</code>                                                                                                                                                  |
| <b>Parâmetro</b>                   | <code>args</code> , e no record <code>nome</code> e <code>carboidratosPor100g</code>                                                                                                      |
| <b>Argumento</b>                   | <code>"Arroz branco cozido"</code> , <code>28.1</code> etc.                                                                                                                               |
| <b>Variável local</b>              | <code>item</code>                                                                                                                                                                         |
| <b>Literal</b>                     | <code>"Arroz branco cozido"</code> , <code>" - "</code> , <code>28.1</code> etc.                                                                                                          |
| <b>Operador</b>                    | <code>=</code> (atribuição), <code>.</code> (acesso), <code>+</code> (concatenação), <code>new</code>                                                                                     |
| <b>Laço</b>                        | <code>for (... : ...)</code>                                                                                                                                                              |
| <b>Bloco</b>                       | Cada par <code>{ }</code>                                                                                                                                                                 |
| <b>Comentário</b>                  | <code>// ...</code>                                                                                                                                                                       |